arXiv:2611.09611v1 [cs.DS] 6 Nov 2026

Online Timely Bin Packing Revisited: A Simpler Algorithm with Optimality in Special Cases

Pei-Lin Chiang* Wing-Kai Hon† Ying-Chia Huang‡ Li-Tzu Yang§

Abstract

Online Timely Bin Packing is a variant of classical online bin packing in which items arrive over time with sizes and deadlines. Packing an item may be deferred until its deadline, but once a collection of items is packed into a bin, the bin is closed and cannot be used again. The objective is to minimize the total number of bins used. The problem was introduced by Khattar in 2023. In 2025, Aminian proposed Discrete-Greedy, a deterministic online algorithm with a competitive ratio of exactly 2, which remains the best-known upper bound. Its analysis is particularly simple; however, the algorithm requires maintaining a fractional simulation internally and may leave usable space in some bins unfilled.

We propose Thrifty-Greedy, a simple deterministic online algorithm that does not require a fractional simulation. We prove that its competitive ratio is exactly 2, matching the best-known upper bound. At the same time, the algorithm automatically achieves the optimal competitive ratio on two natural classes of inputs: it is 1-competitive when all items have the same size, and 1.5-competitive when every item has size in $\{1/3, 2/3\}$. In contrast, Discrete-Greedy has a competitive ratio of at least 2 on both classes.

1 Introduction

In classical online Bin Packing, items arrive one by one, with each item p_i having a size $s_i \in (0, 1]$. Upon arrival, each item must be assigned immediately and irrevocably to a unit-capacity bin, and the objective is to minimize the total number of bins used. Note that the model has no explicit notion of time beyond the order of arrivals.

A classical way to introduce explicit time constraints is Dynamic Bin Packing, introduced by Coffman et al. in 1983 [3]. In this model, each item has an arrival time and a departure time. An arriving item must be assigned immediately to a bin; when the item later departs, the space it occupied becomes available again, while the bin itself remains in the system. This models persistent resources whose capacity can be reused over time, such as cloud servers that remain available while jobs arrive and depart.

In this paper, we study another way to introduce time constraints: online Timely Bin Packing. This problem is introduced by Khattar in 2023 [4]. Here, an arriving item has a deadline, and the algorithm may defer packing it until any time no later than that deadline. When several items are packed together, the entire bin is closed at that time and cannot be used again. Thus, unlike in Dynamic Bin Packing, items do not depart individually and leave reusable space; instead, the bin itself leaves the system.

A motivating example comes from logistics. Items arrive at a warehouse over time and may be stored temporarily before dispatch. Each item has a deadline by which it must be loaded onto a truck or container, and each truck has a fixed capacity. Several items can be dispatched

*Department of Computer Science, National Tsing Hua University, Taiwan. ivylingchiang@gmail.com
†Department of Computer Science, National Tsing Hua University, Taiwan. wkhon@cs.nthu.edu.tw
‡Department of Computer Science, National Tsing Hua University, Taiwan. ereal3478@gmail.com
§Department of Computer Science, National Tsing Hua University, Taiwan. breton0429@gmail.com

together only if they all fit in the truck and are present at the warehouse when the truck departs. Because trucks are costly, the objective is to minimize the number of dispatched trucks. For simplicity, the model assumes that pending items can be stored at the warehouse without limit and at no cost; only the number of dispatched trucks is counted.

Let us now formally define online Timely Bin Packing.

1.1 Problem Definition

We are provided with an unlimited number of bins, each of capacity 1. The input is a finite sequence $I = (p_1, p_2, \ldots, p_n)$ of items. Each item p_i is associated with three parameters: an arrival time a_i, a deadline d_i, and a size $s_i \in (0, 1]$. The items arrive online, meaning that the parameters of an item p_i are revealed only when it arrives at time a_i. We assume that $a_i \leq d_i$ for every item p_i, and call $[a_i, d_i]$ the *interval* of p_i. We index the items in nondecreasing order of arrival time, so that $a_1 \leq a_2 \leq \cdots \leq a_n$. Whenever items have the same arrival time or deadline, ties are broken by item index.

For each item p_i, we need to pack it into some bin at some packing time t_j such that $t_j \in [a_i, d_i]$. Unlike in classical online Bin Packing, where an item must be packed immediately upon arrival, in online Timely Bin Packing, the decision of when to pack an item may be deferred from its arrival until any time no later than its deadline. Yet, once the decision is made, it cannot be changed. An item that has arrived but has not yet been packed is called *pending*.

Formally, given an algorithm $\mathcal{A}$ and an online input sequence I, the output of $\mathcal{A}$ is a collection of bins $O(\mathcal{A}, I) = \{b_1, b_2, \ldots, b_m\}$ satisfying the following conditions:

1. Each bin $b_j \in O(\mathcal{A}, I)$ is associated with a packing time t_j.
2. Each item p_i is assigned to exactly one bin b_j whose packing time satisfies $t_j \in [a_i, d_i]$.
3. For every bin b_j, the total size of the items assigned to b_j is at most 1.

We measure the performance of an online algorithm using standard *competitive analysis*. The cost of an algorithm $\mathcal{A}$ on input I, denoted by $\mathcal{A}(I)$, is the number of bins in its output, i.e., $\mathcal{A}(I) = |O(\mathcal{A}, I)|$. Let OPT denote an optimal offline algorithm that knows the entire input in advance and, for every input I, uses the minimum number of bins. For $c \geq 1$, we say that an online algorithm $\mathcal{A}$ is *c-competitive* if $\mathcal{A}(I) \leq c\,\mathrm{OPT}(I)$ for every input I. The *competitive ratio* of $\mathcal{A}$ is the infimum of all such values c. Our goal is to design an online algorithm $\mathcal{A}$ that minimizes this ratio.

1.2 Prior Work

Khattar introduced online Timely Bin Packing and proved that no deterministic online algorithm can have a competitive ratio below 1.2 [4, Theorem 3]. Khattar also proposed the *Pack-at-Deadline* algorithm and claimed that it is 2-competitive [4, Theorem 1]. Aminian subsequently showed that this claim is incorrect by proving that Pack-at-Deadline has a competitive ratio of at least 2.5 [1, Theorem 4.2.5].

Aminian also improved both the general lower and upper bounds. Aminian proved a lower bound of 1.5 for the competitive ratio of any deterministic online algorithm, even on inputs containing only items of sizes in $\{1/3, 2/3\}$ [1, Theorem 4.5.12]. On the algorithmic side, Aminian introduced *Discrete-Greedy*, a 2-competitive deterministic online algorithm [1, Theorem 4.2.10]. A matching lower bound shows that Discrete-Greedy is no better than 2-competitive, so this analysis is tight [1, Theorem 4.2.11]. Aminian's thesis also studies several other variants, including uniform-size items, unit-length intervals, integer arrival times, randomized algorithms, advice, and the offline problem.

Because Discrete-Greedy is the main benchmark for our work, we next briefly review its definition and analysis.

The fractional relaxation. The definition and analysis of Discrete-Greedy are based on a fractional relaxation of the problem. In online Fractional Timely Bin Packing, an item may be divided into arbitrarily many portions (intuitively, thinking of items as liquids), and different portions may be packed in different bins, as long as every portion is packed at a time within the original item's interval. Let $\mathrm{OPT}_f(I)$ denote the optimal offline cost in this fractional model. Since every feasible integral packing is also a feasible fractional packing, we have

$$\mathrm{OPT}_f(I) \leq \mathrm{OPT}(I).$$

Aminian first designed a simple earliest-deadline-first algorithm that produces an optimal packing in the fractional setting.

> ***Greedy*:** At any time t when at least one pending item reaches its deadline, open a bin and consider all currently available pending items in nondecreasing order of deadline, breaking ties by item index. Pack the items in this order until no item remains or the next item would exceed the remaining capacity. In the latter case, split that item, place exactly the portion needed to fill the bin, and leave the remaining portion pending. Repeat at time t while some pending item has deadline t.

A standard exchange argument shows that Greedy is optimal even among offline fractional packings [1, Lemma 4.2.9]. The argument considers the first bin on which an optimal fractional packing differs from Greedy and exchanges portions of later-deadline items for equal-sized portions of earlier-deadline items, without increasing the number of bins. Repeating the exchange transforms the optimal packing into the one produced by Greedy. Consequently, for every input I,

$$\mathrm{Greedy}(I) = \mathrm{OPT}_f(I) \leq \mathrm{OPT}(I). \tag{1}$$

Aminian then used Greedy to design a simple 2-competitive online algorithm for the integral problem.

> ***Discrete-Greedy*:** Simulate Greedy. For each fractional bin produced by Greedy, pack the items that occur wholly in that bin into one integral bin, omitting fractional portions of items that were split elsewhere. If Greedy ends the fractional bin by splitting an item p, open an additional dedicated bin containing the whole item p and close it at the same time. Future fractional portions of p are omitted from the simulated packing.

Thus, each bin of Greedy projects to at most two bins of Discrete-Greedy. Together with (1), this immediately gives

$$\text{Discrete-Greedy}(I) \leq 2\ \mathrm{Greedy}(I) = 2\ \mathrm{OPT}_f(I) \leq 2\ \mathrm{OPT}(I).$$

The resulting analysis is very simple and straightforward. However, the dedicated-bin rule is deliberately conservative: an item split by the fractional simulation is placed alone, even when it could share a bin with other currently available items. Moreover, the integral algorithm must maintain the state of the fractional simulation, including any remaining fractional portions.

1.3 Our Contributions

The definition of Discrete-Greedy naturally raises the following question: if the next item in earliest-deadline order does not fit in the current bin, why should the algorithm immediately give this item its own bin? A more natural choice is to simply leave that item pending and continue trying to fill the remaining space of the current bin with other items. In this paper, we propose an algorithm based on this simple idea, which we call *Thrifty-Greedy*. Informally, whenever a pending item reaches its deadline, Thrifty-Greedy opens a bin for an earliest-deadline pending item and then scans the other pending items in earliest-deadline order, packing every item that

fits and skipping those that do not. This simple heuristic avoids opening a dedicated bin for the skipped item and also eliminates the need to simulate a fractional packing internally. The formal definition is given in Section 2.

We analyze Thrifty-Greedy in the general setting and in two natural restricted settings. Our results show that this simple heuristic matches the competitive ratio of Discrete-Greedy in the general setting, while attaining the optimal competitive ratio on two restricted classes for which Discrete-Greedy is suboptimal. Notably, these two restricted guarantees are "opportunistic": Thrifty-Greedy does not need to know in advance that the input belongs to either restricted class. The same general-purpose algorithm automatically attains the optimal competitive ratio whenever the input belongs to one of them.

General inputs. In Section 3.2, we show that Thrifty-Greedy is 2-competitive, matching Discrete-Greedy. However, Thrifty-Greedy is not better than Discrete-Greedy in the general setting: we also give a lower-bound construction showing that Thrifty-Greedy is no better than 2-competitive.

The projection argument used for Discrete-Greedy no longer applies directly, because bins of Thrifty-Greedy do not correspond locally to bins of Greedy. Nevertheless, we show that we can still prove the analogous global inequality Thrifty-Greedy$(I) \leq 2$ Greedy(I) by strong induction on the number of items in I. At a high level, we show that we can always select two bins produced by Thrifty-Greedy and remove their entire contents from the input so that the removed items contain the entire contents of at least one bin produced by Greedy. Thus, the number of bins used by Thrifty-Greedy decreases by two, while the number of bins used by Greedy decreases by at least one. Applying the induction hypothesis to the remaining input yields the desired factor of 2.

Uniform-size items. In Section 3.1, we show that Thrifty-Greedy is 1-competitive when all items have the same size, i.e., it uses the same number of bins as any optimal offline algorithm. In fact, on uniform-size inputs, Thrifty-Greedy follows the same strategy as Aminian's *Greedy-on-Deadline* algorithm, which was defined specifically for this class of inputs and was shown to be optimal [1, Observation 4.5.1]. For completeness, we provide a proof using a standard exchange argument. In contrast, Discrete-Greedy has a competitive ratio of at least 2 even on uniform-size inputs; see Appendix A.

Items of sizes in $\{1/3, 2/3\}$. In Section 3.3, we show that Thrifty-Greedy is 1.5-competitive when all items have sizes in $\{1/3, 2/3\}$. This ratio is optimal for the class, since Aminian proved a lower bound of 1.5 for any deterministic online algorithm, even on inputs containing only items of these two sizes [1, Theorem 4.5.12]. Again, in contrast, Discrete-Greedy has a competitive ratio of at least 2 on this restricted class of inputs; see Appendix A.

The proof is rather involved, although the high-level idea is simple. Intuitively, we want to show that the bins produced by Thrifty-Greedy are 2/3-full on average. A 2/3-full or full bin already has sufficient utilization, and pairing a 1/3-full bin with a full bin also gives an average load of 2/3. Thus, the problematic bins are the remaining unpaired 1/3-full bins. If there are x such bins, we would like to argue that an optimal packing must also contain at least x 1/3-full bins. Unfortunately, establishing such a lower bound directly appears difficult.

We therefore give an input-conversion procedure that successively modifies the intervals of some items, converting I into an input I' for which the desired lower bound is easier to establish. The complication is that these modifications may also change the packing produced by Thrifty-Greedy. We must therefore ensure throughout the conversion that Thrifty-Greedy does not use fewer bins on I' than on I. Maintaining this guarantee while creating the structure needed for the lower bound is the main reason the proof becomes technical.

1.4 Other Related Work

Interestingly, the offline version of Timely Bin Packing has been studied from a graph-theoretic perspective. Represent each item p_i by the interval $[a_i, d_i]$ with weight s_i. A set of items can share a bin exactly when their intervals have a common intersection and their total weight is at most 1. Equivalently, a feasible bin corresponds to a clique of total weight at most 1 in the associated weighted interval graph. The offline problem is therefore equivalent to partitioning a weighted interval graph into the minimum number of bounded-weight cliques.

Chen et al. [2] studied this problem under the name *minimum clique partition with constrained bounds in weighted interval graphs*. They proved that it is strongly NP-hard [2, Theorem 1] and cannot be approximated within a factor of $3/2 - \varepsilon$, for any $\varepsilon > 0$, unless $\mathsf{P} = \mathsf{NP}$ [2, Theorem 2]. Myung [7] later gave a 2-approximation algorithm for the case of arbitrary weights, and an exact algorithm for the case of identical weights.

Thus, assuming $\mathsf{P} \neq \mathsf{NP}$, the currently best-known lower and upper bounds on the optimal polynomial-time approximation ratio for Timely Bin Packing are 1.5 and 2, respectively. We remark that the same numerical gap remains in the online setting: the currently best-known lower and upper bounds on the optimal competitive ratio are also 1.5 and 2, respectively.

2 The Proposed Algorithm

In this section, we formally define Thrifty-Greedy. Recall that at any time t, an item is *pending* if it has arrived but has not yet been packed.

> ***Thrifty-Greedy*:** At any time t at which at least one pending item reaches its deadline, repeat the following procedure until no pending item has deadline t.
>
> 1. Choose a pending item p with the earliest deadline (i.e., deadline t), breaking ties by item index. Open a new bin b, assign it packing time t, and pack p into b. We call p the *trigger item* of b.
> 2. Scan the remaining pending items in nondecreasing order of deadline, again breaking ties by item index. Whenever the current item fits in the remaining capacity of b, pack it into b; otherwise, skip it and leave it pending. After every pending item has been considered, close and output b.

For an input sequence of items I, let O denote the corresponding output sequence of bins. For each bin $b \in O$, let $w(b)$ denote the total size of all items packed in b. One reason the proposed algorithm works well is the following lemma.

Lemma 1. *Let b and b' be two distinct bins in O, and let p and p' denote their trigger items, respectively. If $w(b) + w(b') \leq 1$, then the intervals of p and p' do not overlap.*

Proof. Suppose, for contradiction, that the intervals of p and p' overlap. Without loss of generality, assume the deadline of p is within the interval of p'. Then, by the time b is opened at the deadline of p, the item p' is pending. Also, $w(b) + (\text{size of } p') \leq w(b) + w(b') \leq 1$, so by our algorithm, p' would be packed into b. Yet, p' is packed in another bin b', a contradiction. □

3 Performance Analysis

We analyze Thrifty-Greedy in three settings: uniform-size items, general inputs, and items with sizes in $\{1/3, 2/3\}$. The subsections are presented in increasing order of the technical complexity of their analyses.

3.1 Uniform-Size Items

Consider the special case where every item in the input I has the same size. Let $k \in \mathbb{N}$ be the maximum number of items that a bin can hold. We show that Thrifty-Greedy uses the same number of bins as any optimal offline algorithm in this case.

Recall that our proposed algorithm is a *greedy* algorithm, where we will always (i) pick an item p with the earliest deadline, say d_p, to be packed in some bin b, (ii) set b's packing time to d_p, and (iii) add other items to b (prioritized by their deadlines, the earlier the better) as many as possible. To show that our algorithm is optimal, we use the standard exchange argument for greedy algorithms. We first show that such a *greedy choice* is consistent with the choice of some optimal algorithm. Then, inductively, all the greedy choices we have made will be the same as the choices of some optimal algorithm.

Let OPT be an optimal algorithm. Let p denote an item with the earliest deadline in input I, and d_p be its deadline. Let Q be the set of items that are available at time d_p except item p. That is, the arrival time of each item in Q is on or before d_p. Also, let $Q' \subseteq Q$ be the set of $\min(k-1, |Q|)$ items in Q whose deadlines are the earliest, such that these items are exactly those packed in the same bin with p in our greedy algorithm.

Consider how OPT would pack p. Since p has the earliest deadline, without loss of generality, we can assume that OPT sets the packing time of the bin containing p, say b^*, as d_p. We have the following two scenarios:

- If OPT packs p with all the items of Q', done.
- Else, there exists some item $q' \in Q'$ not packed with p by OPT. If the bin b^* containing p still has vacant space for q', we modify the choice of OPT by moving q' into b^*. This change does not increase the number of bins used, so the resulting packing is also optimal; denote the corresponding optimal algorithm by OPT$'$. Repeating the above process, we can put more items of Q' into b^*, while maintaining optimality.

  In the end, either all items of Q' are now in b^*, in which case we are done, or, there exists some $q' \in Q'$ in some bin b other than b^*. For the latter case, there must be some $q \notin Q'$ packed in b^*, and we modify the choices of the optimal algorithm again, by exchanging q and q' in their corresponding bins. Note that such a change is valid, for q' has a deadline earlier than or equal to q, so that q can be moved to b (as its packing time is on or before q'), while q' can be moved to b^* (as its deadline is on or after d_p). Because this exchange does not increase the number of bins used, the resulting packing is still optimal. Repeating the above exchange process, we can put all items of Q' into b^*, while maintaining optimality.

The above discussion shows that there exists some optimal algorithm that makes exactly the same choice as we do, for packing items along with p. Inductively, there exists some optimal algorithm that makes exactly the same choices as we do, for packing each of the items. This gives the following theorem.

Theorem 2. *When all items have the same size,* Thrifty-Greedy$(I) =$ OPT(I) *for every such input* I*.*

3.2 General Inputs

We now consider unrestricted item sizes and prove that Thrifty-Greedy is 2-competitive, with a matching lower bound of 2.

We first prove the upper bound. Recall from (1) that the fractional algorithm Greedy reviewed in Section 1.2 satisfies

$$\text{Greedy}(I) = \text{OPT}_f(I) \leq \text{OPT}(I)$$

for every input I. Therefore, it suffices to establish the following comparison between Thrifty-Greedy and Greedy.

Lemma 3. *For every input I,* Thrifty-Greedy$(I) \leq 2$ Greedy(I).

Proof. We prove the lemma by strong induction on the length of the input I.

Base Case: When $|I| = 1$, the input consists of a single item; both Thrifty-Greedy and Greedy will pack this item into one bin. Thus, Thrifty-Greedy$(I) =$ Greedy$(I) = 1$, and the lemma holds.

Inductive Step: Assume the lemma holds for all inputs of length at most k, for some $k \in \mathbb{N}$. Consider an input I with $|I| = k + 1$. Let us compare the sequences of bins produced by Thrifty-Greedy and Greedy on input I, indexed by their packing orders. We scan the two sequences from left to right to find the first index i, where the contents of their ith bins differ. If no such index exists, the packings are identical, implying Thrifty-Greedy$(I) =$ Greedy(I), so the lemma holds. Otherwise, suppose the packings differ at the ith bin. Since both algorithms pack pending items by increasing order of deadlines, they consider the same set of items in the same order. Therefore, the difference must arise because Greedy can split an item that fits only partially, whereas Thrifty-Greedy cannot.

Specifically, both algorithms pack a set of items C^* into the ith bin. The next item in the priority order, let us call it A, does not fit entirely into the remaining space. Then, the corresponding actions of the two algorithms are as follows.

- Thrifty-Greedy: Cannot split A, so it skips A and continues scanning for other items that fit. Let D^* be the set of subsequent items that Thrifty-Greedy packs into the ith bin. Thus, the ith bin of Thrifty-Greedy contains $C^* \cup D^*$.
- Greedy: Splits A into two pieces, a_1 and a_2. It packs a_1 into the ith bin, making it full. The remainder a_2 is left for subsequent bins. Thus, the ith bin of Greedy contains $C^* \cup \{a_1\}$.

Since Thrifty-Greedy skips A at the ith bin, it must pack A into some later bin, say, the jth bin (where $j > i$). Let B^* denote the set of other items packed alongside A in this jth Thrifty-Greedy bin. Let us now construct a smaller input I' by removing all items contained in the ith and jth Thrifty-Greedy bins from the original input I. Specifically, let $I' = I \setminus (C^* \cup D^* \cup \{A\} \cup B^*)$.

Since $C^* \cup D^*$ and $\{A\} \cup B^*$ constitute the complete contents of two bins in Thrifty-Greedy, removing these items from the input does not affect the packing of the remaining bins (they are packed identically as before). Therefore, the total number of bins used by Thrifty-Greedy on I' decreases by exactly 2. Notice that the length of the new input is strictly shorter than I (i.e., $|I'| \leq k$), hence by the induction hypothesis we have

$$\text{Thrifty-Greedy}(I) \;=\; 2 + \text{Thrifty-Greedy}(I') \;\leq\; 2 + 2 \cdot \text{Greedy}(I').$$

Next, consider the behavior of Greedy. Suppose we modify the original input I by replacing item A with its two split pieces, a_1 and a_2 (where both inherit the same arrival time and deadline as A). Since Greedy operates in the fractional setting and the packing order depends only on arrival times and deadlines, this replacement does not change the original packing. Also, since $C^* \cup \{a_1\}$ constitutes the complete content of one bin in the Greedy packing, we can apply a removal argument similar to the one used for Thrifty-Greedy. Thus, removing those items from the input does not affect the packing of the remaining bins, so that:

$$\begin{aligned} \text{Greedy}(I) &= \text{Greedy}((I \setminus \{A\}) \cup \{a_1, a_2\}) \\ &= 1 + \text{Greedy}(I' \cup D^* \cup \{a_2\} \cup B^*). \end{aligned}$$

Our final step is to show that Greedy$(I' \cup D^* \cup \{a_2\} \cup B^*) \geq$ Greedy(I'). To establish this, we use the following notion of "monotonicity". We say that an input Y *dominates* an input X

if Y can be obtained from X by adding items, increasing their sizes, and/or restricting their intervals. An algorithm $\mathcal{A}$ is *monotone* if $\mathcal{A}(X) \leq \mathcal{A}(Y)$ whenever Y dominates X. Intuitively, adding items, making them larger, or giving them narrower intervals should not make the packing easier.

Although one might expect this property to hold generally, many classical bin packing algorithms are not monotone [5, 6]. Moreover, the standard counterexample for the classical First-Fit heuristic from [6] can be adapted to show that Thrifty-Greedy is not monotone for online Timely Bin Packing.

Consider an input X dominated by Y. Notice that any fractional solution for Y is also a feasible fractional solution for X, so we have $\mathrm{OPT}_f(X) \leq \mathrm{OPT}_f(Y)$. Since Greedy is optimal, we have $\mathrm{Greedy}(X) = \mathrm{OPT}_f(X) \leq \mathrm{OPT}_f(Y) = \mathrm{Greedy}(Y)$. Thus, Greedy is monotone.

Next, we see that I' is dominated by $I' \cup D^* \cup \{a_2\} \cup B^*$. By the monotonicity of Greedy, we have $\mathrm{Greedy}(I' \cup D^* \cup \{a_2\} \cup B^*) \geq \mathrm{Greedy}(I')$. Combining the results in the above discussion, we have

$$\begin{aligned} \text{Thrifty-Greedy}(I) &\leq 2 + 2 \cdot \mathrm{Greedy}(I') \\ &\leq 2 + 2 \cdot \mathrm{Greedy}(I' \cup D^* \cup \{a_2\} \cup B^*) \\ &= 2 \cdot \mathrm{Greedy}(I). \end{aligned}$$

Thus, the lemma holds for $|I| = k + 1$. This completes the inductive step. □

We have thus proven that Thrifty-Greedy is 2-competitive. We now show that the above competitive analysis is indeed tight, i.e., Thrifty-Greedy is not better than 2-competitive.

Consider the following input. Let N be a large integer and let $0 < \varepsilon \leq 1/N$. At time $t = 0$, N items of size $1 - \varepsilon$ arrive with an arbitrarily late deadline. At each time $t = 1, 2, \ldots, N$, two items of size ε arrive: one with an immediate deadline at t, and the other with a deadline at N. Thrifty-Greedy packs the two ε-items together in one bin at each $t = 1, 2, \ldots, N$, and uses another N separate bins for the large $(1-\varepsilon)$-items, using $2N$ bins in total. In contrast, OPT can pack the immediate ε-item with a $(1-\varepsilon)$-item at each $t = 1, 2, \ldots, N$, and pack all N remaining ε-items into a single bin at time $t = N$, using only $N + 1$ bins. As $N \to \infty$, this shows a lower bound of 2 for the competitive ratio of Thrifty-Greedy.

Summarizing, we have the following theorem.

Theorem 4. *In the general setting,* Thrifty-Greedy *is* 2*-competitive, which is tight.*

3.3 Items of Sizes in $\{1/3, 2/3\}$

We now consider the special case where every item has size in $\{1/3, 2/3\}$. We show that Thrifty-Greedy is 1.5-competitive in this setting, which is optimal.

Since every item has size 1/3 or 2/3, we classify the bins used by an algorithm into *small*, *medium*, and *large* bins, which respectively are 1/3-full, 2/3-full, and full. More precisely, let

- x^*, y^*, z^* be the number of small, medium, and large bins used by an optimal offline algorithm OPT on the original input I;
- x, y, z be the number of small, medium, and large bins used by our algorithm Thrifty-Greedy on the original input I;
- x', y', z' be the number of small, medium, and large bins used by our algorithm Thrifty-Greedy on the modified input I'.

At a high level, we would like to show that the bins produced by Thrifty-Greedy are 2/3-full on average, which would give a competitive ratio of 1.5. A medium or large bin already has load at least 2/3, while pairing a small bin with a large bin also gives an average load of 2/3.

Thus, the problematic bins are the remaining unpaired small bins. We would like to show that if Thrifty-Greedy performs badly on such a bin, then we can associate it with a distinct small bin where OPT also performs badly. This would give the desired bound on the competitive ratio. However, the packings produced by Thrifty-Greedy and OPT may be very different, making such a direct association difficult.

In Section 3.1, we handled a similar comparison using a standard exchange argument: we successively modified the packing produced by OPT so that it remained as *good*, while making its choices agree with those of Thrifty-Greedy. Here, we take an orthogonal approach. Instead of modifying either algorithm's packing, we keep the algorithms fixed and successively modify the input I into an input I' so that Thrifty-Greedy remains as *bad* on the modified input, while making it easier to associate the remaining unpaired small bins of Thrifty-Greedy on I' with small bins of OPT on the original input I. By repeatedly applying the conversion, we can compare Thrifty-Greedy on the final modified input I' with OPT on the original input I, and consequently obtain the desired competitive analysis.

In Section 4, we present the input-conversion algorithm and prove its correctness. Specifically, the algorithm converts any input I into another input I' such that the following three properties hold.

- We use the same set of items in I and in I', except some items have more "restrictive" arrival times and deadlines in I'. Since the total size of items remains unchanged, $x^* + 2y^* + 3z^* = x + 2y + 3z = x' + 2y' + 3z'$.
- After pairing small bins with large bins used by Thrifty-Greedy on I', the number of remaining unpaired small bins is at most the number of small bins used by OPT on I. In particular, $x' - z' \le x^*$.
- Our algorithm Thrifty-Greedy performs as badly on I' as on I. That is, Thrifty-Greedy$(I) \le$ Thrifty-Greedy(I').

Remark 5. The first property does not imply the third. Recall from Section 3.2 that I' dominates I, but Thrifty-Greedy is not monotone. Therefore, it is not guaranteed that Thrifty-Greedy$(I) \le$ Thrifty-Greedy(I').

For now, suppose that we have such an input-conversion algorithm. Then, we have the following theorem.

Theorem 6. *When every item has size in* $\{1/3, 2/3\}$, Thrifty-Greedy *is* 1.5*-competitive, which is optimal.*

Proof. We adopt the notation defined above. By combining the equality from the first property with the inequality from the second property, we obtain

$$2x' + 2y' + 2z' \;\le\; 2x^* + 2y^* + 3z^*.$$

Consequently, we have

$$\text{Thrifty-Greedy}(I') \;=\; x' + y' + z' \;\le\; 1.5\,(x^* + y^* + z^*) \;=\; 1.5 \cdot \text{OPT}(I).$$

Combining this with the third property, we obtain

$$\text{Thrifty-Greedy}(I) \;\le\; \text{Thrifty-Greedy}(I') \;\le\; 1.5 \cdot \text{OPT}(I)$$

as desired. Furthermore, our algorithm is optimal, since Aminian proved a lower bound of 1.5 for any deterministic online algorithm, even on inputs containing only items of sizes in $\{1/3, 2/3\}$. This completes the proof. □

4 Input Conversion for the $\{1/3, 2/3\}$ Case

In this section, we will give details on the input conversion mentioned in Section 3.3 to change the input I into another sequence I' satisfying certain properties. Note that the sequence I' is only for analysis purposes, so in the construction of such a sequence, we will assume that some optimal offline algorithm OPT is known, and so is its corresponding output O^* on the input I.

We consider the special case where each item in the input has size either $1/3$ or $2/3$. Here, we adopt the notation defined in Section 3.3. The following are the three properties that I' will be shown to satisfy in this section:

- I and I' use the same set of items, except some items may have more "restrictive" arrival times and deadlines in I'.
- $x' - z' \leq x^*$.
- Thrifty-Greedy$(I) \leq$ Thrifty-Greedy(I').

4.1 The Conversion Algorithm

The formal definition is given in Algorithm 1. In the algorithm, we first fix an optimal offline algorithm OPT and obtain its output O^* on I for reference (Line 1 and Line 2). Then, we initialize I' as I (Line 3), and iteratively update I' by modifying the arrival time and deadline of some selected item. Every time I' is updated, we will rerun our Thrifty-Greedy on I' to obtain the corresponding output O' (Line 4 and Line 10).

The algorithm maintains a time t which is initialized as the latest deadline of all items in I (or the initial I'). This time t will be updated iteratively (Line 9), each time moving to an earlier moment. Based on t, we select the latest small bin in O' with packing time before t, and also its trigger item C (Line 7 and Line 8), and then modify t and some other item's arrival time or deadline based on C, O' and O^*(Line 9). Consequently, the sequence I' is updated.

When no small bins with packing time before t are found, the conversion algorithm terminates, in which case the sequence I' is obtained (Line 11).

Based on C, O' and O^*, the modification in Line 9 is made according to the following three cases:

- Case 1: In O^*, the item C is also in a small bin.
- Case 2: In O', there exists some other bin packed in the interval of C.
- Case 3: Neither Case 1 nor Case 2 applies.

The intuition of this case division is that in the first two cases, we do not even need to modify I', since the output O' is already good enough; therefore, all we need to do is update t. In the remaining case, the small bin containing C in O' may lead to a *bad* performance, so we make some changes to the arrival times or deadlines of some items such that, after the modification, the bin containing C is no longer small. The details of how to perform the modification in each case will be discussed in Section 4.2.

We will show that, after each update, the following invariants hold:

- Invariant 1: I and I' use the same set of items, except some items may have more "restrictive" arrival times and deadlines in I'.
- Invariant 2: For bins with packing time after t, $x' - z' \leq x^*$.
- Invariant 3: Thrifty-Greedy$(I) \leq$ Thrifty-Greedy(I').
- Invariant 4: Time t will be moved to an earlier time slot.

The first three invariants correspond to the three properties mentioned in Section 3.3, except we only require that the original second property holds only for bins packed after t. Invariant 4 guarantees that t will be moved to an earlier time slot after each update, so the conversion algorithm will terminate.

It is easy to verify that all the invariants hold initially, when t is set as the latest deadline of all items. Also, when the conversion terminates, there are no small bins with packing time *before* t, and so the number of small bins x' will not change even if we move t to the earliest moment. That is, Invariant 2 will continue to hold not just for bins after t, but also for all bins. In other words, when the conversion terminates, the sequence I' obtained will satisfy all three properties in Section 3.3.

It remains to describe how each case is handled and to show that the four invariants are maintained after the update.

Algorithm 1 Input Conversion in Special Case

Input: Sequence I
Output: Modified sequence I'
1: Fix any optimal algorithm OPT;
2: $O^* \leftarrow$ output by OPT on I;
3: $I' \leftarrow I$;
4: $O' \leftarrow$ output by Thrifty-Greedy on I';
5: $t \leftarrow$ latest deadline of all items in I';
6: **while** O' contains a small bin with packing time before t **do**
7: $\quad b \leftarrow$ the small bin in O' with latest packing time before t;
8: $\quad C \leftarrow$ trigger item of b;
9: $\quad$ Update t and I' based on C, O' and O^*; ▷ Details in Section 4.2
10: $\quad O' \leftarrow$ output by Thrifty-Greedy on I';
11: **return** I'

4.2 Details of Updates (Line 9)

4.2.1 Case 1

Here, the item C is also in a small bin in O^*. When this happens, we do nothing for the items in I', and just move t to the arrival time of C.

As I' is not changed, Invariants 1 and 3 will hold automatically after the update. Also, t is moved to an earlier moment, so Invariant 4 also holds. To see why Invariant 2 also holds, let us fix some notation. Let t_0 denote the value of t before the update, and t_1 denote the arrival time of C, which is the value of t after the update. Let x'_0 and x'_1 denote the number of small bins packed after t_0 and after t_1 in O', respectively. Similarly, let z'_0 and z'_1 denote the number of large bins packed after t_0 and t_1 in O', and x^*_0 and x^*_1 denote the number of small bins packed after t_0 and t_1 in O^*.

By the choice of C, and the fact (Lemma 1) that the intervals of trigger items of small bins in O' do not overlap, we conclude that $x'_1 = x'_0 + 1$. As t_1 is an earlier moment than t_0, $z'_0 \le z'_1$. Also, C is in a small bin in O^*, so we further have $x^*_1 \ge x^*_0 + 1$. Now, because Invariant 2 holds after t_0, we have $x'_0 - z'_0 \le x^*_0$. Immediately, this implies

$$x'_1 - z'_1 \;\le\; x^*_1,$$

so that Invariant 2 also holds after t_1.

4.2.2 Case 2

Here, some other bin is packed in the interval of C in O'. When this happens, we do nothing for the items in I', and just move t to the arrival time of C.

Similarly to Case 1, we see that Invariants 1, 3 and 4 will hold after the update. To see why Invariant 2 also holds, let us re-use the notation introduced in Case 1. That is,

- t_0 and t_1: the value of t before and after the update;
- x'_0 and x'_1: the number of small bins packed after t_0 and after t_1 in O';
- z'_0 and z'_1: the number of large bins packed after t_0 and t_1 in O'; and
- x^*_0 and x^*_1: the number of small bins packed after t_0 and t_1 in O^*.

Similar to Case 1, we have $x'_1 = x'_0 + 1$. Next, as there is some other bin b packed in the interval of C, such a bin cannot be a small bin nor a medium bin, by Lemma 1. Thus, b is a large bin, so that $z'_1 \geq z'_0 + 1$. As t_1 is an earlier moment than t_0, $x^*_0 \leq x^*_1$. Now, because Invariant 2 holds after t_0, we have $x'_0 - z'_0 \leq x^*_0$. Immediately, this implies

$$x'_1 - z'_1 \ \leq \ x^*_1,$$

so that Invariant 2 also holds after t_1.

4.2.3 Case 3

Here, we consider all those situations that do not match the conditions of Case 1 or Case 2. In this case, we will make modifications to the arrival time of a selected item, and possibly the deadlines of some other items. At any time, we ensure that the following sub-invariants hold:

- **(SI1)** For any item p whose arrival time is modified, its modified arrival time will be after t. Furthermore, it is packed with some item q in O^* whose arrival time is after t.
- **(SI2)** For any item p whose deadline is modified, its modified deadline will be before its original deadline. Furthermore, the modified deadline is on or after t.
- **(SI3)** Let p and q be items that are packed before t by our algorithm Thrifty-Greedy on I'. Then, their time intervals overlap in I if and only if their intervals overlap in I'.

Obviously, the above sub-invariants are true before any item of I is modified. In the following, we assume inductively that they are true before the current modification; based on that, we describe how the modification is made, while ensuring that they continue to hold after the modification, and finally show that the main invariants, Invariants 1, 2, 3, 4, will also hold after the modification.

Our description now begins. First, because we are not in Case 1, item C is not in a small bin in O^*. This implies that C is packed with some other item, say B. Note that the arrival times of both B and C have not been modified, as they are not after t. Also, the time intervals of B and C overlap in I', whether their deadlines have been modified or not (as a modification on deadline can "truncate" only the portion of an interval after t). Then, we have:

Lemma 7. *The arrival time of B is before the arrival time of C. Also, in O', B is packed in some bin before the arrival time of C.*

Proof. Consider O'. Since we are not in Case 2, no other bin is packed in the interval of C. This implies that B is either packed before the arrival time of C, or after the deadline of C. Yet, the latter one is impossible, as otherwise B is available by the time C is packed by our algorithm, so that B would have been put together with C in the same bin. In other words, B *must be* packed before the arrival time of C in O'. Furthermore, B must arrive before C for this to occur. □

Let us look at the bin b' in O' where B is packed. As this bin is packed before the arrival time of C, it must be packed before the deadline of B. So, B is not the trigger item of b'. Instead, there exists some other trigger item, say A. See Figure 1 for the relationship of the intervals of A, B, and C.

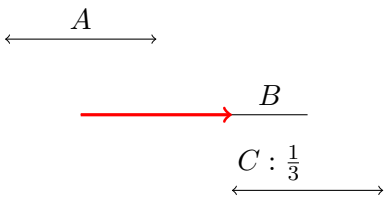


Figure 1: Intervals of items A, B, and C before and after the update.

Let $[a_A, d_A]$ denote the interval of A, and $[a_C, d_C]$ denote the interval of C. Let O'_0 and O'_1 denote the output of our algorithm Thrifty-Greedy on I' before and after the update. Then, the update of I' and t in Case 3 is as follows:

- For each item p (except B) that is packed before a_C in O', if the original deadline is after a_C, we set the deadline to be a value just before a_C, say $a_C^- = a_C - \varepsilon$ for an arbitrarily small positive ε.
- We set the arrival time of B to a_C.
- We set t to a_C^-.

Briefly speaking, the purpose of the above update is to put B and C together, and also "freeze" the bins packed after t, so that they are not affected in the future.

It is easy to see that, after the update, all three sub-invariants (SI1, SI2, and SI3) continue to hold. As for the main invariants, trivially, Invariants 1 and 4 will hold after the update. For Invariants 2 and 3, intuitively, we will show that there is little difference between O'_0 and O'_1, where O'_1 denotes the output of our algorithm Thrifty-Greedy on I' after the update, so that the arrangement of items in their bins can be compared directly. In particular, we consider the items in different *time zones*, and we will show that:

1. **(Z1)** items packed within $[0, d_A)$ in O'_0 are exactly those items packed within the same period $[0, d_A)$ in O'_1. Furthermore, bins are packed in the same way, so that in this period, the number of bins packed in O'_1 is equal to the number of bins packed in O'_0.
2. **(Z2)** items packed within $[d_A, a_C)$ in O'_0, except B, are exactly those items packed within the same period $[d_A, a_C)$ in O'_1. Also, in this period, the number of bins packed in O'_1 is at least the number of bins in O'_0.
3. **(Z3)** In the period $[a_C, d_C]$, O'_0 has exactly one bin, which is packed with C only, while O'_1 also has exactly one bin, which is packed with both B and C (thus, a medium or a large bin).
4. **(Z4)** items packed after d_C in O'_0 are exactly those items packed after d_C in O'_1. Furthermore, bins are packed in the same way, so that in this period, the number of bins packed in O'_1 is equal to the number of bins packed in O'_0.

The above statements imply that after the update, the number of bins in O'_1 is at least the number of bins in O'_0, so that Invariant 3 will hold after the update. Furthermore, let:

- t_0 and t_1: the value of t before and after the update;

- x'_0 and x'_1: the number of small bins packed after t_0 in O'_0 and after t_1 in O'_1;
- z'_0 and z'_1: the number of large bins packed after t_0 in O'_0 and after t_1 in O'_1; and
- x^*_0 and x^*_1: the number of small bins packed after t_0 and t_1 in O^*.

Then, statements Z3 and Z4 imply that

$$x'_1 \;=\; x'_0 \quad \text{and} \quad z'_0 \;\leq\; z'_1.$$

This is because the bin containing C will no longer be small, but may become large after the update, while all the other bins packed after $t_1 = a_C$ are the same in both outputs O'_0 and O'_1. Also, as t_1 is moved to an earlier moment,

$$x^*_0 \;\leq\; x^*_1.$$

As Invariant 2 holds after t_0 in O'_0, we have $x'_0 - z'_0 \leq x^*_0$. Immediately, this implies

$$x'_1 - z'_1 \;\leq\; x^*_1,$$

so that Invariant 2 also holds after the update.

To complete the whole discussion, it remains to show that the above statements for the different time zones are true. Details are as follows.

Proof of Z1: Since the input restricted to the interval $[0, d_A)$ remains unchanged after the update, and by sub-invariant SI3 the overlap relations among these items are preserved, re-running Thrifty-Greedy produces exactly the same bins with packing times in $[0, d_A)$.

Proof of Z2: We first observe the following result.

Lemma 8. *For bins packed within the interval $[d_A, a_C)$ in O'_0, every trigger item remains a trigger item in $[d_A, a_C)$ in O'_1.*

Proof. Let I'_0 and I'_1 denote the sequence I' before and after the update, respectively. Starting from time d_A where we open a bin containing A, since a_B is modified, the bin will no longer contain B, and the items inside may become different. Yet, any items added to the bin before B in O'_0 will still be in the bin in O'_1. Thus, only an item, say p, that is added after B in O'_0 may not be added to the bin now, because another item q is added to the bin instead. Yet, both p and q must have deadline after that of B in I'_0, for otherwise they will be added to the bin before B in O'_0. As the deadline of B in I'_0 is after a_C, the deadline of q is also after a_C, so q cannot be a trigger item in $[d_A, a_C)$. Also, for this to happen, the deadline of p must be after q.

Here, we call q the "culprit" and p a "victim". Note that there can be more than one victim.

As time goes on, we will open a bin for the next item whose deadline is due. The other items, if they exist, in the bin would be the same as in O'_0 in general, but when some of them, say q, becomes a culprit of an earlier bin, the items inside may become different. Yet, any items added to the bin before q in O'_0 will still be in the bin in O'_1 if they are available now. Thus, only an item, say p', that is added after q in O'_0 may not be added to the bin now, because another item q' is added to the bin instead. With the same argument as before, the new culprit q' cannot be a trigger item in $[d_A, a_C)$, p' has deadline after q', and all culprits and victims in the process so far have deadlines after that of B in I'_0.

As time goes on, more and more culprits and victims may be formed. Yet, *none* of them can be a trigger item of any bin in O'_0. Also, as they have deadlines after that of B in I'_0, those deadlines will be after a_C in I'_0; consequently, they cannot be a trigger item in a bin within $[d_A, a_C)$ in O'_0. In other words, only non-trigger items can be culprits or victims, so that they are the only items to be "re-distributed" into a different bin, while all trigger items in O'_0 in $[d_A, a_C)$ must still remain in their own bins, and be trigger items in O'_1. □

By Lemma 8, the number of bins in $[d_A, a_C)$ in O'_1 is at least the number of bins in O'_0. Also, for all items packed during this period in O'_0, their deadlines are now before a_C after the update, so that they will all be packed during this period in O'_1.

Proof of Z3: By statement Z1 and statement Z2, all items with packing time before a_C (except B) are packed before a_C in O'_1, B and C will be the only items left unpacked during $[a_C, d_C]$, so that there will be exactly one bin in O'_1 in this period, containing them.

Proof of Z4: By statements Z1, Z2, and Z3, all items with packing time before d_C are still packed before d_C, so for the time after d_C, all unpacked items are the same in I'_0 and in I'_1. Also, neither their arrival times nor their deadlines are updated. Thus, after re-running Thrifty-Greedy, all the bins with packing time after d_C will be exactly the same.

5 Conclusion

In this paper, we revisited online Timely Bin Packing. We proposed a simple deterministic algorithm with a competitive ratio of exactly 2, matching the best-known upper bound, while simultaneously achieving the optimal competitive ratio for two classes of inputs.

Several interesting problems remain open:

- The optimal competitive ratio for deterministic online algorithms remains between 1.5 and 2. Closing this gap appears challenging.
- This paper focused on deterministic algorithms, but randomization may lead to better performance. Aminian [1, Theorem 4.2.2] proved that, for inputs of length n, no randomized online algorithm can achieve a competitive ratio below $1.5 - \frac{1}{2n-4}$. This lower bound approaches 1.5 as n grows. It remains open whether a randomized online algorithm can achieve a competitive ratio strictly below 2.
- In Timely Bin Packing, pending items may be stored temporarily without cost and without any limit on storage capacity. It would be interesting to study variants with bounded storage or with holding costs for pending items.

AI Methodology

ChatGPT was used to assist with literature searches and with improving the authors' existing text.

References

[1] Aida Aminian. "Algorithms for Timely Bin Packing, Fair TCP Acknowledgement, and Variants". Master's Thesis. York University, 2025.

[2] Mingxia Chen, Jianbo Li, Jianping Li, Weidong Li, and Lusheng Wang. "Some Approximation Algorithms for the Clique Partition Problem in Weighted Interval Graphs". In: *Theoretical Computer Science* 381.1–3 (2007), pp. 124–133. DOI: 10.1016/j.tcs.2007.04.030.

[3] E. G. Coffman Jr., M. R. Garey, and D. S. Johnson. "Dynamic Bin Packing". In: *SIAM Journal on Computing* 12.2 (1983), pp. 227–258. DOI: 10.1137/0212014.

[4] Zhadyra Khattar. "Online Timely Bin Packing". Master's Thesis. Utrecht University, 2023.

[5] F. D. Murgolo. *On the Monotonicity of Certain Bin Packing Algorithms*. Tech. rep. 244. Department of Information and Computer Sciences, University of California, Irvine, 1984.

[6] F. D. Murgolo. "Anomalous Behavior in Bin Packing Algorithms". In: *Discrete Appl. Math.* 21.3 (1988), pp. 229–243. DOI: 10.1016/0166-218X(88)90069-8.

[7] Young-Soo Myung. "On the Clique Partitioning Problem in Weighted Interval Graphs". In: *Theoretical Computer Science* 396.1–3 (2008), pp. 290–293. DOI: 10.1016/j.tcs.2008.02.016.

A Lower Bounds for Discrete-Greedy

The Discrete-Greedy algorithm works by simulating the optimal fractional algorithm, Greedy, such that each bin packed by Greedy is projected to no more than two integral bins.

For example, suppose Greedy packs the bins b_1, b_2, b_3, b_4, where the last item p of b_1 is split between b_1 and b_3, and the last item q of b_3 is split between b_3 and b_4. Then, Discrete-Greedy produces: b_1 as before but without the fraction of p; a dedicated bin b'_1 for p; b_2 as before; b_3 as before but without the fractions of p and q; a dedicated bin b'_3 for q; and b_4 as before but without the fraction of q.

The projection step makes the analysis for the competitive ratio of Discrete-Greedy extremely simple. However, the projection step also leads to inefficiency. As we now show, Discrete-Greedy has a competitive ratio of at least 2, even for two simple classes of inputs.

A.1 Uniform-Size Items

We construct a sequence of inputs I_n such that all items in I_n have the same size (for each n) and Discrete-Greedy$(I_n) \to 2\,\mathrm{OPT}(I_n)$ as $n \to \infty$, showing that Discrete-Greedy's competitive ratio is at least 2 for this class of inputs.

Let ε be a real number such that $0 < \varepsilon \leq \frac{1}{n(n+1)(n+2)}$. The input I_n consists of items with arrival time 0 and size $s = \frac{1}{n+1} + \varepsilon$. The items are divided into $n+1$ groups:

- Groups $1, \ldots, n$: Each group contains $n+1$ items. Group i has deadline i.
- Group $n+1$: Contains n items with deadline $n+1$.

For Greedy, items are sorted by increasing deadlines. Since $(n+1)s > 1$, a single bin cannot hold all items of one group. Hence, Greedy fills the first n bins $b_1, \ldots, b_n$ completely. Due to the small ε, the total size of all items is no more than $n+1$, so Greedy uses $n+1$ bins. Notice that since the cumulative size of the first k groups exceeds k by a small margin $k(n+1)\varepsilon < s$, each bin $b_1, \ldots, b_n$ ends with a split item.

Consequently, Discrete-Greedy projects each of the bins $b_1, \ldots, b_n$ into 2 integral bins (one for the non-split items, one for the split item that closes the bin) and projects b_{n+1} into 1 integral bin. Thus, Discrete-Greedy uses $2n+1$ bins.

For OPT, each bin holds exactly n items (since $ns < 1 < (n+1)s$). The total number of items is $n(n+1) + n = n(n+2)$. Thus, OPT can pack all items into $\lceil \frac{n(n+2)}{n} \rceil = n+2$ bins.

A.2 Items of Sizes in $\{1/3, 2/3\}$

Similarly, we construct a sequence of inputs I_n such that all items have size either $1/3$ or $2/3$, and Discrete-Greedy$(I_n) \to 2\,\mathrm{OPT}(I_n)$ as $n \to \infty$, showing that Discrete-Greedy's competitive ratio is at least 2 for this class of inputs.

The input I_n consists of items with arrival time 0. The items are divided into $n+1$ groups, ordered by increasing deadlines as follows:

- Group 1: Two items of size $2/3$.
- Groups $2, \ldots, n$: Each group contains one item of size $1/3$ followed by one item of size $2/3$.

- Group $n+1$: One item of size $1/3$.

For Greedy, the items are packed in the order specified above.

- The first bin b_1 packs the first item of Group 1 (size $2/3$) and the first $1/3$ of the second item. It ends with a split item.
- For $k = 2, \ldots, n$, the bin b_k starts with the remaining $1/3$ of the split item from Group $k-1$, packs the item of size $1/3$ from Group k, and fills the remaining space with the first $1/3$ of the item of size $2/3$ from Group k. Thus, each bin b_k is full and ends with a split item.
- The final bin b_{n+1} starts with the remaining $1/3$ of the split item from Group n and packs the single item of size $1/3$ from Group $n+1$. The bin is not full but contains no split items at the end.

In total, Greedy uses $n+1$ bins. Since bins $b_1, \ldots, b_n$ all end with a split item, Discrete-Greedy projects each of them into 2 integral bins (one for the non-split item, one for the split item that closes the bin). The final bin b_{n+1} projects to 1 integral bin. Thus, Discrete-Greedy uses $2n+1$ bins.

For OPT, we can pair the items. There are $n+1$ items of size $2/3$ and n items of size $1/3$. Thus, OPT uses $n+1$ bins.